Spring Dependency Injection - Deep Handbook

Built from the actual code and the full learning journey.

Chapter 1 - App.java

Original Code

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("spring.xml");

Dev obj = (Dev)context.getBean(Dev.class);
obj.Build();
```

Your Question

Why do we need obj if Spring already created a Dev object?

Wrong Mental Model

```
getBean() creates another Dev object.
```

Correct Mental Model

```
Spring already created Dev earlier.

Dev d = new Dev();

getBean() only returns a reference.

Dev obj = d;
```

Takeaway

One Dev object. Multiple references are possible.

Chapter 2 - Interface Reference

Original Code

```
private Computer comp;
```

Your Question

Why doesn't Java create a Computer object?

Wrong Mental Model

```
Computer comp -> create Computer object
```

Correct Mental Model

```
Computer is an interface.

Computer comp;

creates only a reference variable.

Memory:

comp -> null
```

Takeaway

Interface type is known during compilation. No object is created.

Chapter 3 - Setter Injection

Original Code

```
public void setComp(Computer laptopObj){
    this.comp = laptopObj;
}
```

Your Question

Who is injecting what?

Wrong Mental Model

```
Computer object is injected.
```

Correct Mental Model

```
Spring creates a Laptop object.

laptopObj -> Laptop Object

this.comp = laptopObj

After assignment:

comp -----------\
                 -> Laptop Object
laptopObj ------/
```

Takeaway

Spring injects a Laptop reference into Dev.comp.

Chapter 4 - Why Laptop Can Be Passed

Original Code

```
setComp(new Laptop());
```

Your Question

Computer is an interface. Why is Laptop accepted?

Wrong Mental Model

```
Only Computer objects can be passed.
```

Correct Mental Model

```
Laptop implements Computer.

Therefore:

Laptop IS-A Computer

Java allows:

Computer comp = new Laptop();
```

Takeaway

Interfaces allow many implementations.

Chapter 5 - XML Wiring

Original Code

```
<property name="comp" ref="laptopBean"/>
```

Your Question

Why are there two names?

Wrong Mental Model

```
Both comp refer to the same thing.
```

Correct Mental Model

```
name='comp'
    -> Dev property

ref='laptopBean'
    -> bean to inject

Equivalent:

dev.setComp(laptopBean);
```

Takeaway

Left side = property. Right side = bean.

Chapter 6 - Polymorphism

Original Code

```
comp.compile();
```

Your Question

Why does Laptop.compile() run?

Wrong Mental Model

```
Computer.compile() should run.
```

Correct Mental Model

```
Reference Type = Computer

Actual Object = Laptop

comp ----> Laptop Object

Java checks the actual object type.
```

Takeaway

Runtime polymorphism uses actual object type.

Chapter 7 - Loose Coupling

Original Code

```
private Laptop comp;
```

Your Question

Why is Computer better than Laptop?

Wrong Mental Model

```
Directly using Laptop is fine forever.
```

Correct Mental Model

```
Tight Coupling:

private Laptop comp;

Loose Coupling:

private Computer comp;

Now implementation can change.
```

Takeaway

Dev depends on abstraction, not implementation.

Complete Execution Trace

```
main()
 ↓
ClassPathXmlApplicationContext("spring.xml")
 ↓
Spring reads XML
 ↓
Creates Laptop Bean
 ↓
Laptop Constructor
 ↓
Creates Dev Bean
 ↓
Dev Constructor
 ↓
setComp(laptopBean)
 ↓
this.comp = laptopBean
 ↓
Stores Beans
 ↓
getBean(Dev.class)
 ↓
obj.Build()
 ↓
working on a project
 ↓
comp.compile()
 ↓
Laptop.compile()
 ↓
working on a project on laptop
```

Final Aha Moments

1. Computer comp; does not create an object.
2. Interfaces are contracts.
3. Only Laptop/Desktop objects exist.
4. this.comp = laptopObj is the actual injection point.
5. getBean() returns an existing bean.
6. XML acts like a switchboard.
7. Spring creates and wires objects.
8. Spring Boot uses annotations but the same concepts.
9. Reference Type != Actual Object Type.
10. Loose Coupling exists because Dev depends on Computer.